SAFARI LANKA

Member-wise Design Patterns for the 6 Functional Areas

A practical, viva-friendly guide based on the supplied SLIIT design-pattern lab and lecture slides. Each member receives at least one simple pattern that fits their own Safari Lanka function, with Java code structure, justification, implementation path and a short viva answer.

Patterns selected	Factory • Strategy • Observer • Decorator
Existing real pattern	Payment Strategy (already implemented in the current project)
Goal	Give every member a simple, defensible design pattern for the viva.

1. Source basis from your SLIIT materials

This guide uses only the simple patterns covered in the three materials you provided. The Lab 06 sheet teaches the Strategy Pattern and asks the group project to incorporate an appropriate design pattern. Lecture Part I covers Singleton and Observer. Lecture Part II covers Strategy, Factory and Decorator.

Source	Relevant pattern content	How it is used here
Lab 06 - Design Patterns	Strategy: strategy interface, concrete strategies, context; group-project pattern requirement.	Used as the main template for Strategy implementations and viva wording.
Lecture 8 - Design Patterns Part I	Design-pattern classifications plus Singleton and Observer.	Observer is used for vehicle-maintenance notifications. Singleton is listed as an optional system-wide extra.
Design Patterns Part II	Strategy, Factory and Decorator, including payment, object creation and wrapper examples.	Used for booking identity factory, package pricing, trip assignment, payment and permit validation.

Important
The patterns below are chosen for simplicity and a close match to each member's function. Patterns do not need to be unique across members. Using the same Strategy idea in different functional areas is valid when the behaviors genuinely vary.

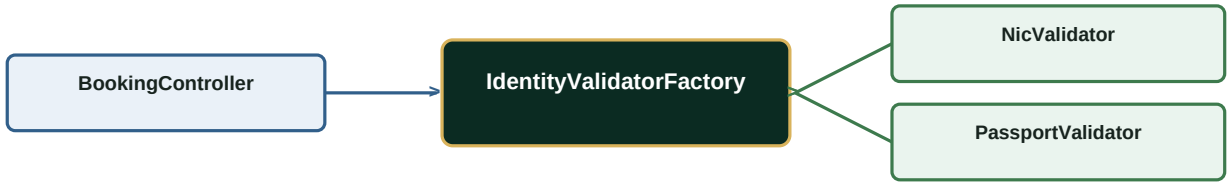
2. Member-wise pattern allocation and strength ranking

Rank	Member / Function	Pattern	Fit	Status
1	M5 - Payment, Invoice, Cancellation & Refund	Strategy	★★★★★ VERY STRONG	Already implemented
2	M2 - Package, Route, Time Slot & Pricing	Strategy	★★★★★ VERY STRONG	Recommended
3	M6 - Park Entry, Permit Validation & Compliance	Decorator	★★★★■ STRONG	Recommended
4	M3 - Driver & Vehicle Maintenance	Observer	★★★★■ STRONG	Recommended
5	M4 - Guide Roster & Trip Assignment	Strategy	★★★★■ STRONG	Recommended
6	M1 - Tourist Booking, Visitor Group & Permit Application	Factory	★★★★■ STRONG	Recommended

MEMBER 1	Tourist Booking, Visitor Group & Permit Application	Factory Pattern Creational	STRONG

Why Factory fits this function

Tourist registration and booking already need different identity-validation behaviors. Sri Lankan tourists may use NIC validation, while other tourists use passport validation. A Factory keeps the controller from manually creating the correct validator with repeated if/else object-creation code.



Suggested backend location

`backend/src/main/java/com/safarilanka/service/identity/`

Simple Java implementation

```
public interface IdentityValidator {
    boolean isValid(String value);
}

public class NicValidator implements IdentityValidator {
    public boolean isValid(String value) {
        return value != null &&
               (value.matches("\\d{12}") ||
                value.matches("\\d{9}[Vv]"));
    }
}

public class PassportValidator implements IdentityValidator {
    public boolean isValid(String value) {
        return value != null &&
               value.matches("[A-Za-z0-9]{6,20}");
    }
}

public class IdentityValidatorFactory {
    public static IdentityValidator create(String type) {
        if ("NIC".equalsIgnoreCase(type))
            return new NicValidator();
        if ("PASSPORT".equalsIgnoreCase(type))
            return new PassportValidator();
        throw new IllegalArgumentException("Invalid identity type");
    }
}
```

Viva answer

“I used the Factory Pattern because the booking module must create the correct identity validator for NIC or Passport. The controller depends on the factory and the common validator interface, instead of directly creating concrete validator classes.”

Benefits: cleaner controller, centralized object creation, easy to add another identity type later.

MEMBER 2	Safari Package, Route, Time Slot & Pricing	Strategy Pattern Behavioral	VERY STRONG

Why Strategy fits this function

Package pricing is an algorithm that can vary. Safari Lanka already calculates prices based on visitor count and extra vehicles. Strategy separates each pricing rule into its own class and lets the pricing context choose the correct rule without a long conditional method.

Suggested backend location

`backend/src/main/java/com/safarilanka/service/pricing/`

Simple Java implementation

```
public interface PricingStrategy {
    double calculate(double pricePerPerson,
                     int visitors,
                     double extraVehicleFee);
}

public class SingleVehiclePricingStrategy
        implements PricingStrategy {
    public double calculate(double price, int visitors,
                            double extraVehicleFee) {
        return price * visitors;
    }
}

public class MultiVehiclePricingStrategy
        implements PricingStrategy {
    public double calculate(double price, int visitors,
                            double extraVehicleFee) {
        int vehicles = (int) Math.ceil(visitors / 6.0);
        return (price * visitors) +
               ((vehicles - 1) * extraVehicleFee);
    }
}

public class PricingContext {
    private PricingStrategy strategy;

    public PricingContext(PricingStrategy strategy) {
        this.strategy = strategy;
    }

    public double calculate(double price, int visitors,
                            double extraVehicleFee) {
        return strategy.calculate(
            price, visitors, extraVehicleFee);
    }
}
```

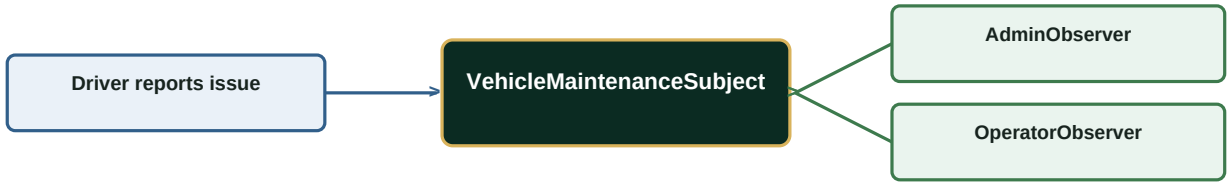
Viva answer

“I used Strategy because pricing has multiple algorithms. A booking with one vehicle uses one strategy, while a booking requiring multiple vehicles uses another. Both implement the same PricingStrategy interface, so pricing rules can change without changing the booking controller.”

MEMBER 3	Jeep Driver & Vehicle Maintenance	Observer Pattern Behavioral	STRONG

Why Observer fits this function

When a driver reports maintenance or a vehicle changes to MAINTENANCE/AVAILABLE, multiple parts of the system may need the same update. Observer lets the maintenance subject notify Admin and Safari Operations automatically without tightly coupling those receivers to the maintenance logic.



Suggested backend location

`backend/src/main/java/com/safarilanka/service/maintenance/observer/`

Simple Java implementation

```
public interface MaintenanceObserver {
    void update(String vehicleNo, String status);
}

public class AdminMaintenanceObserver
        implements MaintenanceObserver {
    public void update(String vehicleNo, String status) {
        System.out.println(
            "Admin: " + vehicleNo + " -> " + status);
    }
}

public class OperatorMaintenanceObserver
        implements MaintenanceObserver {
    public void update(String vehicleNo, String status) {
        System.out.println(
            "Operator: " + vehicleNo + " -> " + status);
    }
}

public class VehicleMaintenanceSubject {
    private final List<MaintenanceObserver> observers
        = new ArrayList<>();

    public void addObserver(MaintenanceObserver o) {
        observers.add(o);
    }

    public void notifyObservers(
            String vehicleNo, String status) {
        for (MaintenanceObserver o : observers)
            o.update(vehicleNo, status);
    }
}
```

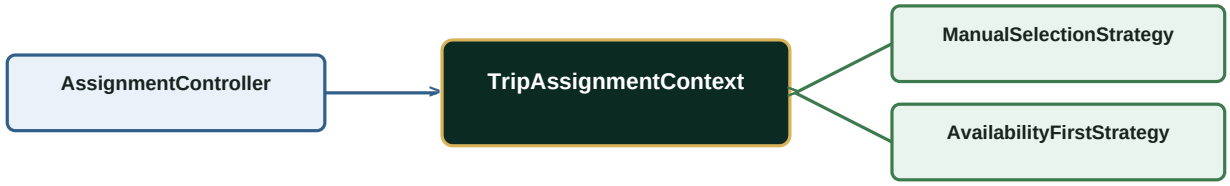
Viva answer

“I used Observer because one maintenance-status change must be communicated to more than one interested component. The subject keeps a list of observers and calls update() whenever the vehicle status changes.”

MEMBER 4	Safari Guide Roster & Trip Assignment	Strategy Pattern Behavioral	STRONG

Why Strategy fits this function

Trip assignment can use different selection rules. One rule can respect the operator's manual choice; another can automatically choose from available resources. Strategy keeps assignment selection logic separate from AssignmentController and makes the rule interchangeable.



Suggested backend location

`backend/src/main/java/com/safarilanka/service/assignment/`

Simple Java implementation

```
public interface AssignmentStrategy {
    User chooseDriver(List<User> drivers);
}

public class ManualSelectionStrategy
        implements AssignmentStrategy {
    private final Long selectedDriverId;

    public ManualSelectionStrategy(Long id) {
        this.selectedDriverId = id;
    }

    public User chooseDriver(List<User> drivers) {
        return drivers.stream()
            .filter(d -> d.getId().equals(selectedDriverId))
            .findFirst()
            .orElseThrow();
    }
}

public class AvailabilityFirstStrategy
        implements AssignmentStrategy {
    public User chooseDriver(List<User> drivers) {
        return drivers.stream()
            .filter(User::isAvailable)
            .filter(d -> !d.isOnLeave())
            .findFirst()
            .orElseThrow();
    }
}
```

Viva answer

“I used Strategy because the system can select drivers using different assignment rules. The AssignmentController works with the AssignmentStrategy interface, so we can switch between manual selection and automatic availability-first selection.”

MEMBER 5	Payment, Invoice, Cancellation & Refund	**Strategy Pattern** Behavioral	VERY STRONG / IMP LEMENTED

Why this is the strongest current pattern

This pattern already exists in the current Safari Lanka project. Card payment and bank transfer are different payment algorithms that share one PaymentStrategy contract. PaymentContext delegates processing to the selected strategy. This directly matches the payment example used in the supplied Strategy lecture.

Existing backend location

`backend/src/main/java/com/safarilanka/service/payment/`

Current project structure (simplified)

```
public interface PaymentStrategy {
    Payment process(PaymentStrategyCommand command);
}

@Component
public class CardPaymentStrategy
        implements PaymentStrategy {
    @Override
    public Payment process(
            PaymentStrategyCommand command) {
        // validate card request
        // create payment
        // mark successful card payment as PAID
        return payment;
    }
}

@Component
public class BankTransferPaymentStrategy
        implements PaymentStrategy {
    @Override
    public Payment process(
            PaymentStrategyCommand command) {
        // save bank transfer / PDF slip
        // keep payment PENDING
        // Finance Manager later approves or rejects
        return payment;
    }
}

@Component
public class PaymentContext {
    private final List<PaymentStrategy> strategies;

    public Payment process(PaymentStrategy strategy,
                           PaymentStrategyCommand command) {
        return strategy.process(command);
    }
}
```

Viva answer
“I used Strategy because Safari Lanka supports different payment methods with different processing rules. CardPaymentStrategy and BankTransferPaymentStrategy implement the same PaymentStrategy interface, and PaymentContext executes the selected behavior.”

Why it is ranked #1: It is already implemented as explicit pattern classes, closely matches the lecture's e-commerce payment example, and is very easy to demonstrate in code.

MEMBER 6	Park Entry, Digital Permit Validation & Compliance	Decorator Pattern Structural	STRONG

Why Decorator fits this function

A digital permit must pass several independent checks: basic permit validity, payment approval, complete driver/guide assignment and gate-entry conditions. Decorator is a clean match because each check wraps the previous validator and adds one more responsibility without changing the base validator class.

Suggested backend location

`backend/src/main/java/com/safarilanka/service/permit/validation/`

Simple Java implementation

```
public interface PermitValidator {
    void validate(Booking booking);
}

public class BasicPermitValidator
        implements PermitValidator {
    public void validate(Booking booking) {
        if (booking == null)
            throw new IllegalStateException("Booking missing");
    }
}

public abstract class PermitValidatorDecorator
        implements PermitValidator {
    protected final PermitValidator next;

    protected PermitValidatorDecorator(
            PermitValidator next) {
        this.next = next;
    }
}

public class PaymentCheckDecorator
        extends PermitValidatorDecorator {
    public PaymentCheckDecorator(PermitValidator next) {
        super(next);
    }

    public void validate(Booking booking) {
        next.validate(booking);
        if (booking.getPaymentStatus()
                != PaymentStatus.PAID)
            throw new IllegalStateException(
                "Payment not approved");
    }
}

public class AssignmentCheckDecorator
        extends PermitValidatorDecorator {
    public AssignmentCheckDecorator(PermitValidator next) {
        super(next);
    }

    public void validate(Booking booking) {
        next.validate(booking);
        // verify driver, guide and linked vehicle
    }
}
```

Viva answer
“I used Decorator because permit validation contains several checks that can be added one by one. Each decorator wraps the same PermitValidator interface and adds another validation rule without changing the base validator.”

3. Final member-wise summary

Member	Function	Pattern	One-line justification
1	Tourist Booking / Visitor / Permit Application	Factory	Creates the correct NIC or Passport validator without direct concrete-object creation in the controller.
2	Package / Route / Time Slot / Pricing	Strategy	Pricing algorithm changes based on one-vehicle vs multi-vehicle booking rules.
3	Driver / Vehicle Maintenance	Observer	Admin and Operator can react automatically when maintenance or vehicle status changes.
4	Guide Roster / Trip Assignment	Strategy	Driver/guide selection can use manual or availability-first algorithms.
5	Payment / Invoice / Cancellation / Refund	Strategy	Card and bank transfer processing are interchangeable payment strategies; already implemented.
6	Park Entry / Digital Permit / Compliance	Decorator	Adds payment, assignment and gate validation checks around a common permit validator.

4. Recommended viva order

Start with Member 5 (Payment Strategy) because it is already implemented and directly matches the payment example in the Strategy lecture. Then show Member 2 and Member 4 Strategy applications, followed by Member 1 Factory, Member 3 Observer and Member 6 Decorator.

Simple rule for every member
Explain four things only: **Problem** → **Pattern** → **Main classes** → **Benefit**. This follows the lecture idea that a design pattern has a name, problem, solution and consequences.

5. Optional system-wide extra pattern

Singleton can be used for a central SystemSettingsRegistry if your group wants one additional pattern. The supplied Lecture Part I defines Singleton as a class with one instance and a global access point, implemented using a private constructor, a static instance and getInstance(). It is not required for the six-member allocation above.

6. Source notes

• Lab 06 - Design Patterns: Strategy Pattern structure and group-project task.
• Week11 Lecture DesignPattern - Part 01: design-pattern classification, Singleton and Observer.
• DesignPattern Part II: Strategy, Factory and Decorator, including e-commerce payment and coffee decorator examples.

Implementation note: Member 5's Payment Strategy is already present in the current Safari Lanka codebase. The other five sections in this document are simple implementation-ready additions designed to fit the existing project structure without changing the user-visible workflow.